

Maestría en Inteligencia Artificial Aplicada

Actividad 3
Código de experimentación base con MLflow

Alumno: Jordan Piero Borda Colque

Matrícula: A01840539

Correo: A01840539@tec.mx

Curso: TC5061: Operaciones de Aprendizaje Automático (MLOps)

Profesor titular: Iván Reyes Amezcua

Fecha: 11 de octubre de 2026

1. Repositorio y ejecución

- **Repositorio:** `https://github.com/neronapier/TC5061-A03-Experimentacion-MLflow`
- **MLflow (solo lectura):** `https://161.132.54.170`
- **Experimento:** `TC5061_A03_AdultIncome_LogReg` (10 corridas)

El modelo base de la Actividad 2 (regresión logística sobre UCI Adult, para predecir si una persona gana más de 50K USD al año) pasó del notebook a `train.py`. El notebook `notebooks/eda_adult.ipynb` conserva solo la exploración de datos. Las corridas se ejecutaron en el VPS del curso, en Docker: MLflow 3.16.1 con artefactos en MinIO. Comandos exactos:

```
pip install -r requirements.txt
python train.py                                    # valores por defecto
python train.py --C 1 --l1-ratio 0 --class-weight balanced --seed 42 --run-name 07_l2_C1_balanced
./scripts/run_experimentos.sh                      # las 10 corridas
# En el VPS (MLFLOW_TRACKING_URI=http://mlflow:5000 dentro del contenedor):
docker compose --profile run run --rm -T -w /work/actividad3 -e PY=python runner \
    bash scripts/run_experimentos.sh
```

2. Implementación

Estructura. Cada función de `train.py` tiene una sola responsabilidad: `parse_args`, `fijar_semilla`, `cargar_datos` (descarga UCI si falta), `limpiar` (decisiones de la Actividad 2), `construir_modelo` (`Pipeline` con preprocesamiento y clasificador), `evaluar`, `guardar_artefactos` y `main`, que une todo dentro de `mlflow.start_run()`.

Parámetros por línea de comandos (`argparse`, con valores por defecto y ayuda en `--help`): `--C` (1.0), `--l1-ratio` (0 = L2, 1 = L1), `--class-weight` (`none`/`balanced`), `--min-frequency` (50), `--max-iter` (1000) y `--seed` (42), además de `--test-size`, `--cv-folds`, `--experiment-name` y `--run-name`.

Registro en MLflow. `log_param` para cada argumento, el solver y los tamaños de entrenamiento y prueba; `log_metric` para 7 métricas de prueba (accuracy, balanced accuracy, precision, recall, F1, ROC-AUC y PR-AUC), la ROC-AUC de validación cruzada (media y desviación), la ROC-AUC de entrenamiento y el número de coeficientes distintos de cero; `log_artifact` para la matriz de confusión con la curva ROC, el reporte de clasificación y `metrics.json`; y `mlflow.sklearn.log_model` para el pipeline, con firma y ejemplo de entrada.

Reproducibilidad. La semilla fija `random`, `numpy` y `PYTHONHASHSEED`, y se pasa como `random_state` a la división, a la validación cruzada y al modelo. Las versiones están fijadas en `requirements.txt`. La corrida 10 repite la 01 con los mismos argumentos y da las mismas métricas hasta el último decimal. Ese mismo resultado también se obtuvo en un entorno local nuevo instalado desde `requirements.txt`.

3. Corridas

Tabla 1: Corridas del experimento (métricas en el conjunto de prueba, $n = 9\,758$).

Corrida	C	l1	class_w	mf	seed	ROC-AUC	F1	Prec.	Recall	Coef.
01 baseline	1	0	none	50	42	0.9024	0.6592	0.7195	0.6083	74
02 L2 fuerte	0.001	0	none	50	42	0.8954	0.5984	0.7449	0.5000	74
03 L2 débil	100	0	none	50	42	0.9025	0.6582	0.7175	0.6079	74
04 L1	0.01	1	none	50	42	0.8981	0.6382	0.7205	0.5728	19
05 L1 fuerte	0.001	1	none	50	42	0.8909	0.5470	0.7727	0.4234	6
06 Elastic Net	0.1	0.5	none	50	42	0.9024	0.6544	0.7188	0.6006	50
07 balanced	1	0	balanced	50	42	0.9023	**0.6715**	0.5572	**0.8450**	74
08 min_freq 500	1	0	none	500	42	0.9023	0.6583	0.7202	0.6062	50
09 otra semilla	1	0	none	50	7	0.8993	0.6383	0.7140	0.5771	72
10 repetición	1	0	none	50	42	0.9024	0.6592	0.7195	0.6083	74

4. Análisis de resultados

Mejor configuración. En ROC-AUC, que no depende del umbral, hay un empate: las corridas 01, 03, 06, 07 y 08 quedan entre 0.9023 y 0.9025. Esa diferencia es menor que la desviación estándar de la validación cruzada (0.004). Por eso elijo la **07 (L2, $C = 1$, `class_weight=balanced`)**: con la misma ROC-AUC obtiene el mejor F1 (0.672) y la mejor balanced accuracy (0.817), y sube el recall de la clase `>50K` de 0.61 a 0.85. Su costo es menos precisión (0.56) y menos accuracy (0.80), porque predice `>50K` con más frecuencia.

Hiperparámetro más influyente. `class_weight` es el que más cambia las métricas que dependen del umbral: mueve el recall 24 puntos sin cambiar la ROC-AUC, así que en la práctica desplaza el umbral de decisión. `C` solo influye con regularización fuerte: con $C = 0.001$ la ROC-AUC baja a 0.895 (L2) y 0.891 (L1), y el recall cae a 0.50 y 0.42; entre $C = 1$ y $C = 100$ no hay diferencia. L1 sirve para simplificar el modelo: con $C = 0.01$ usa 19 de 74 coeficientes y conserva una ROC-AUC de 0.898. `min_frequency` (de 50 a 500) quita 24 columnas y no cambia el desempeño.

Limitaciones.

- Se usa una sola partición de prueba. Cambiar solo la semilla (corrida 09) mueve el F1 en 0.021 y la ROC-AUC en 0.003, más que la diferencia entre las mejores configuraciones.
- Las corridas se eligieron a mano (10 configuraciones). Faltan una búsqueda sistemática y el ajuste del umbral, que daría un efecto parecido a `balanced`.
- Las corridas L1 y Elastic Net usan `saga` y las L2 usan `lbfgs`, así que el solver también cambia entre ellas.
- Se probó un solo modelo lineal y no se evaluó el sesgo por `sex` y `race`, que el EDA mostró relevante.

A. Evidencias de MLflow

Las figuras se tomaron de la interfaz pública `https://161.132.54.170`, donde se pueden revisar todas las corridas.

Experiments › TC5061_A03_AdultIncome_LogReg ›

Training runs

Views

metrics.rmse < 1 and params.model = "tree" | All time | State: Active | Datasets | Sort: undefined | Columns | Group by | New run

				Metrics						Parameters				
Run Name	Created	Duration	n_coef_no_cero	test_accuracy	test_f1	test_precision	test_recall	test_roc_auc	C	class_weight	l1_ratio	min_frequency	seed
01_baseline_l2_C1	14 minutes ago	1.1min	74	0.8494568…	0.6592437…	0.7194936…	0.6083047…	0.9024253…	1.0	none	0.0	50	42
02_l2_C0.001	12 minutes ago	25.9s	74	0.8393113…	0.5983606…	0.7448979…	0.5	0.8954401…	0.001	none	0.0	50	42
03_l2_C100	12 minutes ago	1.1min	74	0.8488419…	0.6581691…	0.7175341…	0.6078767…	0.9024551…	100.0	none	0.0	50	42
04_l1_C0.01	11 minutes ago	22.4s	19	0.8445378…	0.6382065…	0.7205169…	0.5727739…	0.8980615…	0.01	none	1.0	50	42
05_l1_C0.001	10 minutes ago	22.8s	6	0.8321377…	0.5470132…	0.77265625	0.4233732…	0.8908917…	0.001	none	1.0	50	42
06_elasticnet_C0.1	10 minutes ago	40.9s	50	0.8481246…	0.6543843…	0.71875	0.6005993…	0.9023899…	0.1	none	0.5	50	42
07_l2_C1_balanced	9 minutes ago	57.8s	74	0.8021110…	0.6715427…	0.5571549…	0.8450342…	0.9023187…	1.0	balanced	0.0	50	42
08_l2_C1_minfreq500	8 minutes ago	50.0s	50	0.8493543…	0.6582984…	0.7202441…	0.6061643…	0.9023048…	1.0	none	0.0	500	42
09_baseline_seed7	7 minutes ago	57.8s	72	0.8434105…	0.6382575…	0.7139830…	0.5770547…	0.8992989…	1.0	none	0.0	50	7
10_baseline_repeticion	6 minutes ago	58.7s	74	0.8494568…	0.6592437…	0.7194936…	0.6083047…	0.9024253…	1.0	none	0.0	50	42

Figura 1: Tabla comparativa de las 10 corridas en el experimento `TC5061_A03_AdultIncome_LogReg` (parámetros y métricas de prueba).

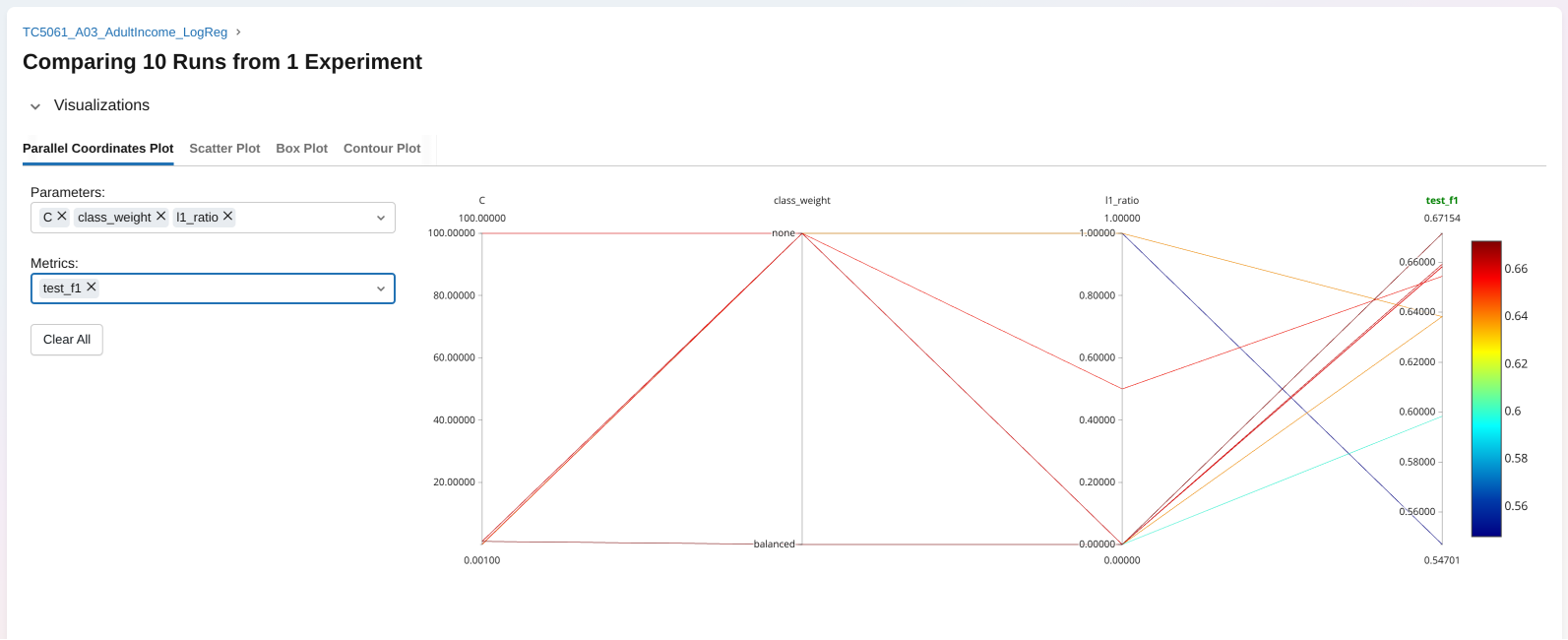


Figura 2: Vista *Compare runs*: coordenadas paralelas de `C`, `class_weight` y `l1_ratio` contra `test_f1`.

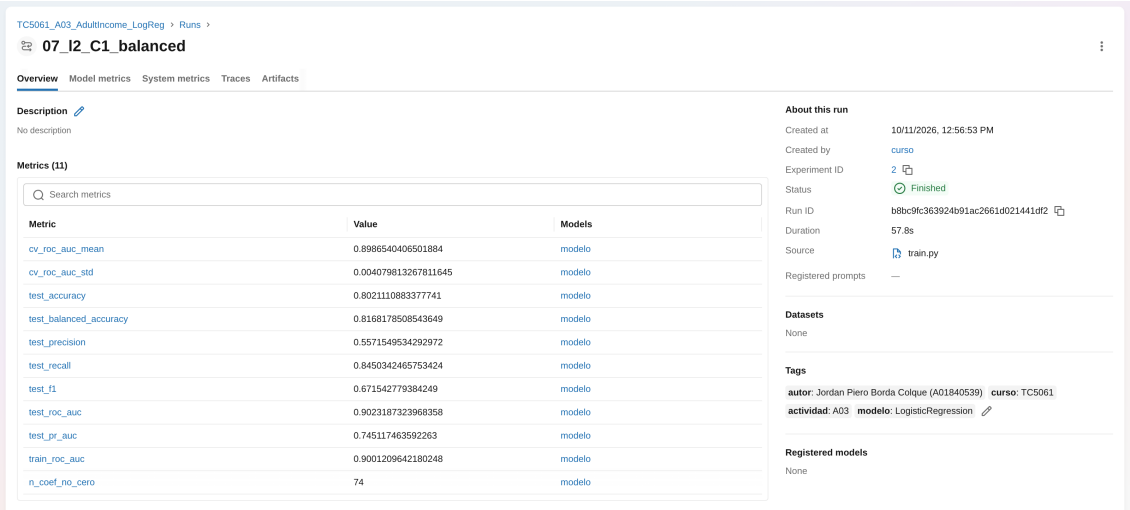


Figura 3: Detalle de la corrida `07_l2_C1_balanced`: métricas, tags y modelo asociado.

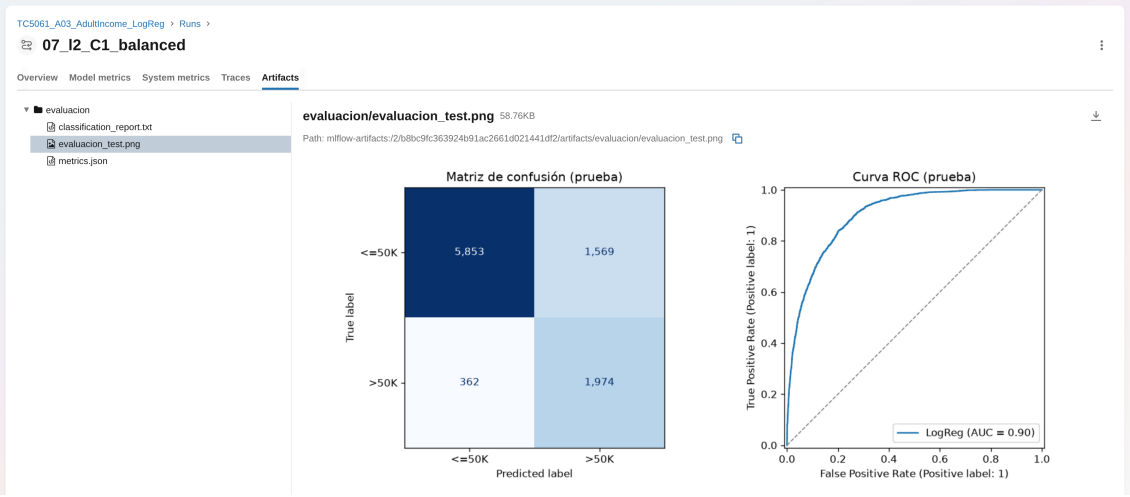


Figura 4: Artefactos de la corrida `07_l2_C1_balanced`: reporte, `metrics.json` y la imagen con la matriz de confusión y la curva ROC.

Referencias

Becker, B., & Kohavi, R. (1996). *Adult* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20

Huyen, C. (2022). *Designing machine learning systems: An iterative process for production-ready applications*. O'Reilly Media.

Lauchande, N. (2021). *Machine Learning Engineering with MLflow*. Packt Publishing.

MLflow. (s.f.). *MLflow Tracking* [Documentación]. https://mlflow.org/docs/latest/ml/tracking/

Python Software Foundation. (s.f.). *argparse* [Documentación]. https://docs.python.org/3/library/argparse.html